

DIAMOND

Real-Time Anomaly Detection and Autonomous Execution on Prediction Market Microstructure

Perry Jenkins | March 2026 | Internal Technical Reference

1. Abstract

DIAMOND is a live autonomous trading system operating on Kalshi, a CFTC-regulated binary options exchange (prediction market). The system ingests a real-time trade stream via WebSocket, constructs rolling microstructural profiles per contract, and evaluates each incoming trade against a ten-dimensional anomaly detection feature space. Composite scores are formed via capped weighted aggregation with dead-feature redistribution. Scores exceeding a calibrated threshold trigger order placement through an order-book-aware execution engine that adapts aggressiveness to conviction tier. An event-aware conviction tracker prevents self-contradictory positions across sibling contracts using exponentially decayed signal accumulation. A self-learning pipeline performs post-hoc feature attribution and Bayesian weight adaptation from settlement outcomes, with an optional ML scoring layer (L1-regularized logistic regression or gradient-boosted trees, selected via expanding-window time-series cross-validation on Brier score).

2. System Architecture

The system is structured as a single-process asynchronous event loop (Python asyncio) with four concurrent coroutines: (i) a WebSocket consumer ingesting the Kalshi trade channel, (ii) a REST poller refreshing order book snapshots and market metadata at 30-second and 5-minute intervals respectively, (iii) a settlement/fill polling loop operating on a 2-minute cadence, and (iv) a Dash-based HTTP dashboard serving real-time visualizations on port 8080. Persistence is handled by SQLite with schema migration, pruning (30-day trades/anomalies, 2-day book snapshots), and indexed queries for cross-market correlation. The architecture avoids multi-process complexity; all state is shared in-memory with SQLite as the durable backing store.

A 180-second warmup period after cold start suppresses alert dispatch and trade placement while rolling profiles populate, preventing false-positive cascades from empty baselines. HTTP sessions are recycled every 30 minutes to prevent stale-connection failures observed in long-running (multi-day) deployments on OCI compute.

3. The Feature Space: $\mathbb{R}^{10}$ Anomaly Representation

Each incoming trade is mapped to a point in $\mathbb{R}^{10}$ where each coordinate is a normalized anomaly score in $[0, 1]$. The features are designed to capture distinct microstructural signatures of informed trading. All features incorporate minimum-activity gates (sample size thresholds, depth floors) to suppress spurious signals in thin or nascent markets.

3.1 Trade Size Z-Score

Given a rolling mean μ and standard deviation σ of trade sizes per contract, the z-score $z = (x - \mu) / \sigma$ is mapped to $[0, 1]$ via a rational sigmoid:

$$f(z) = 1 - 1/(1 + z/2) \text{ for } z > 0, \text{ else } 0$$

This provides a smooth, monotonically increasing mapping: $z=1$ maps to 0.33, $z=2$ to 0.50, $z=3$ to 0.60, $z=4$ to 0.67. The rational form (as opposed to the standard logistic sigmoid) was chosen for heavier tails -- extreme z-scores contribute meaningfully without saturating prematurely. Returns zero when no baseline exists ($\mu = 0$), enforcing the principle that anomaly detection requires a null model.

3.2 Volume Spike Ratio

Compares trailing 1-hour contract volume V_{1h} against the historical hourly average $\bar{V}$. A dead zone at ratio ≤ 1.5 suppresses normal variance. Above this, scoring follows a logarithmic compression:

$$f(r) = \ln(r) / \ln(15) \text{ for } r > 1.5$$

The base-15 logarithm normalizes such that a 15x volume spike maps to 1.0. This logarithmic scaling reflects Weber-Fechner-like perception: each multiplicative doubling of volume contributes roughly equal informational surprise. The feature captures sustained flow anomalies on the hourly timescale, complementing the instantaneous trade-velocity feature (F8).

3.3 Order Book Imbalance

Measures the normalized depth asymmetry of the limit order book:

$$I = |D_{\text{yes}} - D_{\text{no}}| / (D_{\text{yes}} + D_{\text{no}})$$

where D_{yes} and D_{no} are the total resting quantities on each side. Requires minimum aggregate depth of 200 contracts (thin books produce noise). Because Kalshi prediction markets are structurally directional (most events have a natural favorite), only extreme imbalance $I > 0.7$ fires, rescaled linearly over $[0.7, 1.0]$ to $[0, 1]$. This high threshold was calibrated empirically to suppress the ~60% of markets with natural lean > 0.5 .

3.4 Taker Side Skew

Examines the fraction of aggressive (taker-initiated) volume on each side within the trailing 1-hour window. With R_{yes} as the yes-taker ratio, the deviation metric is:

$$d = 2 * |R_{\text{yes}} - 0.5|$$

Only fires when $d > 0.8$ (corresponding to $R_{\text{yes}} > 0.90$ or $R_{\text{yes}} < 0.10$) with a minimum of 50 trades in the window. Requires extreme one-sided aggression. This is the highest-weighted feature ($w = 0.20$) because persistent directional aggression is the strongest microstructural fingerprint of informed flow in binary markets.

3.5 Price Impact

Measures the absolute price displacement induced by a single trade, in cents. Prices on Kalshi are bounded in $[1, 99]$ cents. The score scales linearly: a 30-cent move maps to 1.0, with a 2-cent dead zone for normal tick fluctuations. This feature has low weight (0.05) because large price impacts are rare on Kalshi and often reflect illiquidity rather than information.

3.6 Cross-Market Correlation

Counts distinct tickers exhibiting concentrated anomalies (≥ 3 anomalies per ticker) within a 60-second window. Fires only when ≥ 15 such tickers are active, yielding a high bar that prevents positive feedback loops where one anomaly cascade triggers cross-market correlation signals which in turn generate more anomalies. Weight is minimal (0.03) as the feature is prone to systematic correlation during high-activity periods (e.g., Super Tuesday, March Madness tipoffs).

3.7 Sweep Detection

Detects aggressive price-level sweeps: a taker consuming resting liquidity across consecutive price levels within 60 seconds. The algorithm tracks the longest monotone subsequence of prices on the same taker side. Score scales as $(\text{levels} - 1) / 4$, with a 1.2x speed bonus when inter-trade arrival time averages < 2 seconds. This captures the signature of an informed participant who needs immediate size and is willing to walk the book -- the microstructural equivalent of a market order with unlimited slippage tolerance.

3.8 Trade Velocity

Measures the ratio of trades in the last 30 seconds to the 24-hour baseline rate ($\text{trades per 30s} = \text{trade_count_24h} / 2880$). Uses the same logarithmic compression as volume spike but on a shorter timescale, catching burst activity from algorithmic participants or panic-driven retail flow. Dead zone at 3x; base-20 log normalization.

3.9 Size Concentration (Whale Detection)

Computes the fraction of 5-minute volume attributable to the top-3 largest trades. In diffuse markets, volume is distributed across many small trades; when the concentration ratio C_3 exceeds 0.5, it suggests a single large participant ("whale") is active. Rescaled linearly over $[0.5, 1.0]$ to $[0, 1]$. Requires ≥ 10 trades in the window to suppress noise from low-activity contracts.

3.10 Book Pressure Delta

Measures the rate of change of order book imbalance over a 5-minute lookback. Whereas static imbalance (F3) captures the current state, this feature captures the derivative -- a large shift indicates active repositioning (building a wall or pulling liquidity). The signed imbalance at each snapshot is:

$$I(t) = (D_{\text{yes}}(t) - D_{\text{no}}(t)) / (D_{\text{yes}}(t) + D_{\text{no}}(t))$$

$$\text{delta} = |I(t) - I(t - 300s)|$$

Fires when $\text{delta} > 0.10$, rescaled over $[0.10, 0.45]$ to $[0, 1]$. This is analogous to tracking the time derivative of an order parameter in a phase transition: the system is not anomalous because the book is imbalanced, but because the imbalance is changing rapidly.

4. Composite Scoring with Capped Redistribution

The composite anomaly score is a weighted linear combination of the ten features with a redistribution mechanism for inactive (zero-valued) features:

$$S_{\text{raw}} = \sum_i w_i * f_i(x) \quad W_{\text{active}} = \sum_{\{i : f_i > 0\}} w_i \quad R = \min(W_{\text{total}} / W_{\text{active}}, 1.5) \quad S = S_{\text{raw}} * R$$

The redistribution factor R rescales the raw score to account for features that cannot fire (e.g., no order book data available). This is capped at 1.5x (or 1.3x when fewer than 3 features fire) to prevent two low-weight features from inflating a score to CRITICAL. The cap ensures that reaching the ALERT threshold ($S \geq 0.55$) requires either 3+ moderate features or 2+ high-weight features firing simultaneously. This design choice directly encodes the prior that isolated anomalies are less informative than concordant multi-signal events.

Feature Weight Vector (March 2026)

Feature	Weight	Empirical Fire Rate at ALERT	Rationale
Taker Side Skew	0.20	70%	Strongest informed-flow fingerprint
Sweep Detection	0.15	16%	Selective, high-conviction
Trade Size Z-Score	0.12	94%	Baseline; not differentiating
Book Imbalance	0.12	9.6%	Very strong when fires (avg 0.91)
Trade Velocity	0.12	28%	Extremely strong signal (avg 0.98)
Volume Spike	0.10	99%	Baseline; not differentiating
Book Pressure Delta	0.06	--	Directional derivative of F3
Price Impact	0.05	1.6%	Rare; often illiquidity artifact
Size Concentration	0.05	--	Weak signal, needs more data
Cross-Mkt Correlation	0.03	--	Feedback-loop prone

Table 1. Feature weights tuned via firing-rate analysis on live data (March 22, 2026).

5. Alert Classification and Execution Tiers

The composite score S in $[0, 1]$ is discretized into four alert levels with associated system actions. The thresholds were calibrated to yield an empirical distribution of approximately 86% NONE, 11% LOG, 2% NOTABLE, 1% ALERT, with CRITICAL at roughly 1-3 events per day during active market hours:

Level	Threshold	Action	Execution Parameters
LOG	$S \geq 0.25$	SQLite record only	--
NOTABLE	$S \geq 0.45$	macOS notification	--
ALERT	$S \geq 0.55$	Pushover + live order	ask + 1c, max spread 15c
CRITICAL	$S \geq 0.78$	Emergency push + order	ask + 3c, max spread 25c

6. Order-Book-Aware Execution Engine

When a composite score exceeds the ALERT threshold, the execution engine places a limit order on the Kalshi REST API. The pricing algorithm is adaptive to conviction tier:

- **Book interrogation:** A fresh order book snapshot is fetched (not the stale periodic snapshot) to determine the current best ask. For a YES buy, $\text{best_ask} = 100 - \text{highest_no_bid}$, exploiting the complementary price structure of binary contracts.
- **Cross margin:** The limit price is set at $\text{best_ask} + M$ cents, where M in $\{1, 2, 3\}$ increases with conviction tier. This ensures fills in thin markets while bounding adverse selection cost.
- **Spread filter:** Orders are rejected if the spread between signal price and best ask exceeds a tier-dependent maximum (15-25 cents), preventing execution in dislocated or stale markets.
- **Time-in-force:** All orders are GTC (good-till-canceled) rather than IOC (immediate-or-cancel). Kalshi markets are sufficiently thin that IOC orders were observed to expire unfilled in $> 80\%$ of cases. Stale GTC orders are automatically cancelled after 1 hour.

All positions are held to settlement (no early exits except conviction flips), providing a clean binary outcome (win/loss) for unbiased signal quality measurement. A kill switch halts new trade placement when cumulative daily P&L drops below $-\$20$, implementing a hard drawdown constraint.

7. Event-Aware Conviction Tracking

Kalshi events (e.g., a basketball game) spawn multiple sibling contracts (Team A wins, Team B wins). Taking opposing positions across siblings is structurally guaranteed to lose (you pay two spreads for a net-zero payoff). The conviction tracker prevents this via exponentially decayed signal accumulation per event.

Event grouping uses the ticker prefix: `ticker.rsplit('-', 1)[0]` extracts the shared event identifier. For each event, the tracker maintains a time-tagged signal history. The conviction for a given ticker within an event is the decayed sum:

$$C(\text{ticker}, t) = \sum_i s_i * \exp(-(t - t_i) / \tau)$$

where s_i is the composite anomaly score of the i -th signal, t_i its timestamp, and $\tau = 420$ seconds (7 minutes) is the decay time constant. Note that τ here is the e-folding time, not the half-life ($t_{1/2} = \tau * \ln(2) \sim 291\text{s}$). After one τ , a signal retains $1/e \sim 37\%$ of its original weight; after $2*\tau$, $\sim 13.5\%$.

This formulation naturally handles the persistence-vs-intensity tradeoff: three moderate alerts (score 0.65) at the same recency contribute roughly the same conviction as a single strong alert (score $0.90 * 2.2 \sim 1.95$ vs $0.65 * 3 = 1.95$). Fresh intensity wins over stale persistence as old signals decay, with a single tuning knob (τ) controlling the crossover timescale.

Decision Logic

When a new signal arrives for ticker T in event E , the tracker computes $C(T, t)$ and finds the strongest opposing ticker T' with conviction $C(T', t)$. The decision is:

ALLOW: no prior signals for other tickers in E FLIP: $C(T, t) > C(T', t) + \delta$
BLOCK: otherwise ($\delta = 0.4$)

The flip threshold $\delta = 0.4$ represents the estimated round-trip cost of exiting an existing position and entering the new one (spread cost + adverse selection). A FLIP triggers an exit order on the old position before entering the new one.

8. Portfolio Risk Controls

Five independent gates constrain the execution engine to prevent concentration risk and tail scenarios:

Gate	Constraint	Rationale
Position limit	Max 100 concurrent open	Capital allocation bound
Category exposure	Max 15 per category	Sector concentration limit
Event cap	Max 1 per event	Structural loss prevention
Burst throttle	Max 8 trades / 5 min	Cascade protection
Kill switch	Halt at -\$20 daily P&L	Hard drawdown constraint

9. Self-Learning Pipeline

Once sufficient settlement data accumulates (≥ 50 trades), a retrospective analysis pipeline computes feature-level attribution and proposes adaptive weight updates.

9.1 Feature Attribution

For each settled trade, the system identifies which features were active ($\text{score} > 0$) and which was the primary driver (highest-scoring feature). Per-feature metrics are computed: win rate, average P&L, primary-driver win rate, and mean feature score conditional on outcome (wins vs losses). Co-occurrence patterns (sorted feature-set tuples) are also tracked to identify which feature combinations are predictive.

9.2 Adaptive Weight Update

Optimal weights are derived from a reliability score per feature:

$$\text{rho}_i = \max(0, \text{WR}_i - 0.40) * \text{sqrt}(n_i)$$

where WR_i is the empirical win rate and n_i the trade count for feature i . The 0.40 baseline penalty means features below 40% win rate contribute zero weight (they are actively harmful). The $\text{sqrt}(n)$ term provides a Bayesian-flavored confidence scaling: low-sample features are down-weighted regardless of apparent win rate. Reliability scores are normalized to sum to 1.0, then blended 50/50 with the current weights to prevent wild swings:

$$w'_i = 0.5 * w_i + 0.5 * (\text{rho}_i / \sum_j \text{rho}_j)$$

This exponential moving average over weight updates provides stability while allowing the system to adapt to changing market regimes on a timescale of ~ 100 settled trades.

10. Machine Learning Scoring Layer

An optional ML scorer operates in parallel with the hand-tuned composite, initially in shadow mode (logging predictions without acting) for A/B comparison.

10.1 Feature Engineering

The raw 9-dimensional feature vector (cross-market correlation excluded as a dead feature) is augmented with 6 engineered features: number of features firing, maximum feature score, entry price, side indicator,

composite-price interaction term, and target-encoded category (with Bayesian smoothing: $P(\text{win}|\text{cat})$ blended with global prior using smoothing parameter $m=10$, following the standard credibility formula).

10.2 Null Importance Testing

Before training, features undergo a permutation-based null importance test. For each of 100 iterations, labels are shuffled and an L1-regularized logistic regression is fit. A feature is retained only if its real importance ($| \text{coefficient} |$) exceeds the 95th percentile of its null distribution. This controls the family-wise false discovery rate at approximately 5% per feature, preventing the model from fitting to noise dimensions.

10.3 Tiered Model Selection

Two candidate models are evaluated via expanding-window time-series cross-validation (respecting temporal ordering -- no future leakage):

- **Tier 1 (Lasso):** L1-regularized logistic regression ($C=0.5$, SAGA solver, balanced class weights). Serves as the baseline; its sparsity-inducing penalty automatically zeros noise features.
- **Tier 2 (GBM):** Gradient-boosted decision trees (80 estimators, depth 3, learning rate 0.05, 80% subsample/feature bagging). Only evaluated when $n \geq 150$ samples. Must beat Lasso by > 0.005 Brier score to be selected -- a practical significance threshold preventing complexity without benefit.

The winning model is wrapped in Platt scaling (CalibratedClassifierCV with sigmoid method, 3-fold internal CV) to produce calibrated probability estimates $P(\text{win})$.

10.4 Edge Prediction

The final prediction is not a raw probability but an **edge** estimate:

$$\text{edge} = P_{\text{calibrated}}(\text{win}) - P_{\text{market}}(\text{win})$$

where $P_{\text{market}} = \text{entry_price} / 100$ is the market-implied probability (binary contracts settle at 0 or 100 cents). Positive edge implies the model believes the trade is mispriced in our favor. This formulation directly estimates the quantity relevant for Kelly criterion sizing (edge / odds), enabling future position sizing optimization.

11. Mathematical Properties and Design Rationale

11.1 Signal-to-Noise in Binary Markets

Prediction markets present a distinct signal detection problem compared to traditional equity microstructure. In a binary contract, the terminal payoff is Bernoulli-distributed with $P(\text{payout}=100) = p$, $P(\text{payout}=0) = 1-p$. The market price is an estimator of p . Informed trading shifts the true p while the market price lags, creating a transient alpha that decays as the order book absorbs the information. The features in DIAMOND are designed to detect this transient before the book fully adjusts.

The key insight is that in binary markets, the *direction* of informed flow is trivially observable (yes-taker vs no-taker), unlike in equity markets where the direction of institutional flow requires sophisticated techniques to infer. The challenge is distinguishing informed flow from noise, which is why the feature space emphasizes intensity (z-scores, velocity ratios) and concordance (multiple features firing simultaneously) rather than direction per se.

11.2 Exponential Decay as a Memory Kernel

The conviction tracker's exponential decay $\exp(-t/\tau)$ is the simplest causal kernel with a single characteristic timescale. It is the Green's function of the first-order ODE $dC/dt = -C/\tau + S(t)$, where $S(t)$ is the signal input -- i.e., the conviction acts as a leaky integrator of anomaly signals. This is equivalent to an exponential moving average with $\alpha = dt/\tau$, providing a natural connection to the EMA-based signal processing literature. The 7-minute timescale was chosen to match the observed decorrelation time of Kalshi order flow: signals older than ~15 minutes (2τ) carry < 14% weight and are effectively forgotten.

11.3 Brier Score as Model Selection Criterion

The ML pipeline selects models by Brier score rather than AUC or accuracy. For a calibrated probability forecaster, the Brier score decomposes as:

$$BS = \text{Reliability} - \text{Resolution} + \text{Uncertainty}$$

This decomposition (Murphy, 1973) rewards both discrimination (resolution) and calibration (reliability), making it the natural proper scoring rule for systems where the predicted probability directly determines position sizing. AUC, by contrast, is invariant to monotone transformations and ignores calibration entirely -- a model with perfect AUC but poor calibration would systematically over- or under-bet.

12. Deployment and Operational Considerations

The system runs 24/7 on an Oracle Cloud Infrastructure compute instance (Ubuntu, Python 3.13). Key operational mechanisms include: (i) automatic WebSocket reconnection with re-authentication on 401 errors, (ii) 30-minute HTTP session recycling to prevent stale connections that cause silent order placement failures after multi-day uptime, (iii) force-recycling on connection errors (ClientConnectorError, ServerDisconnectedError), and (iv) Pushover push notifications for all order failures, kill switch activations, and conviction flips, ensuring no operational failure goes undetected.

Deployment is via scp (not git) using a shell script that syncs Python files and optionally restarts both the monitor and dashboard processes. The dashboard provides real-time visualization of feature radar charts, volume/anomaly timelines, portfolio P&L, and a filterable anomaly feed with 30-second auto-refresh.

13. Summary of Contributions

DIAMOND demonstrates that microstructural anomaly detection can be profitably applied to prediction markets -- a domain with distinct statistical properties from traditional equities (binary payoffs, structural directionality, event-driven correlation). The system combines: (1) a principled 10-dimensional feature space capturing distinct aspects of informed flow, (2) a capped-redistribution composite scoring mechanism that encodes concordance priors, (3) an exponentially-decayed conviction tracker solving the multi-contract arbitrage prevention problem, (4) a tiered ML pipeline with null importance testing, proper time-series cross-validation, and Platt-calibrated edge prediction, and (5) a layered risk management framework with five independent gates. The entire system operates autonomously in a single async process, processing the live trade stream with sub-second latency from signal detection to order placement.